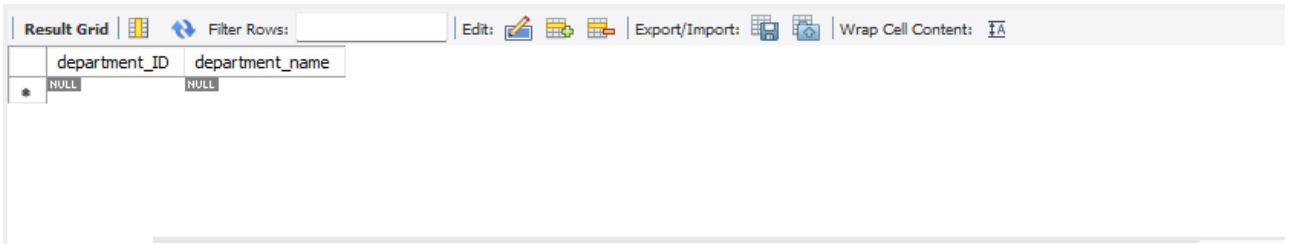
Assignment 5 – DDL Commands & Constraints

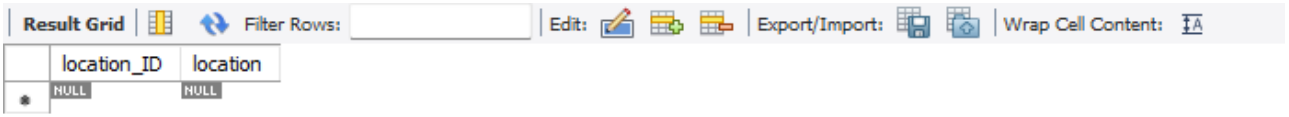
DDL Commands :

1. Table Creation (CREATE): Created a database name ‘employess_db’ and created tables base on the schema:

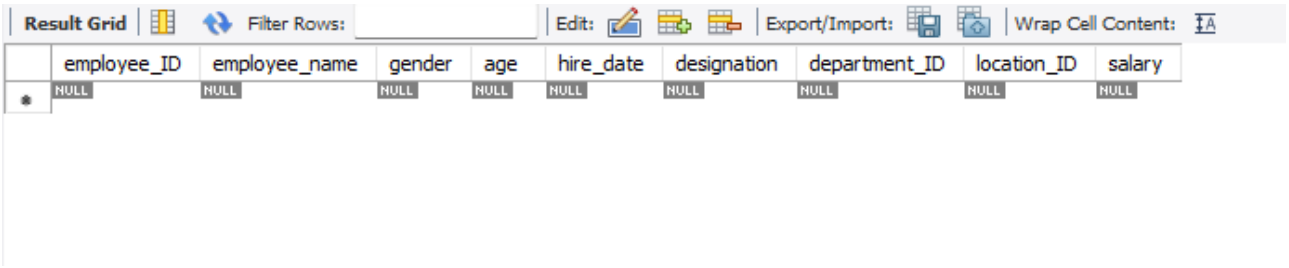
- Departments:



- Locations:

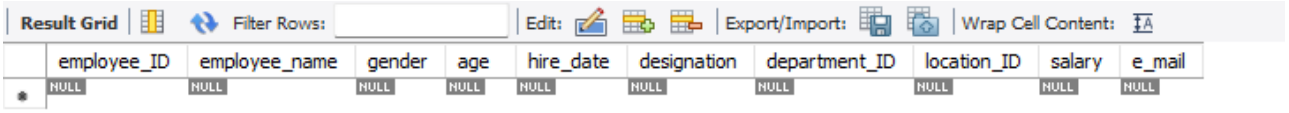


- Employees:



2. Table Alteration (ALTER): Consider the following scenarios and write the SQL statements to alter the structure of the tables accordingly:

- Used ALTER table employees ADD COLUMN(e_mail VARCHAR(50)); to add new email column to the employees table.



- Modified datatype of designation column from varchar(100) to varchar (500) using ALTER and MODIFY
  ALTER table employees MODIFY designation VARCHAR(500);
- Age column dropped from employees table using DROP command

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_ID	employee_name	gender	hire_date	designation	department_ID	location_ID	salary	e_mail
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Rename hire date column to date of joining using ALTER and RENAME command

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_ID	employee_name	gender	date_of_joining	designation	department_ID	location_ID	salary	e_mail
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3.Table Renaming (RENAME):

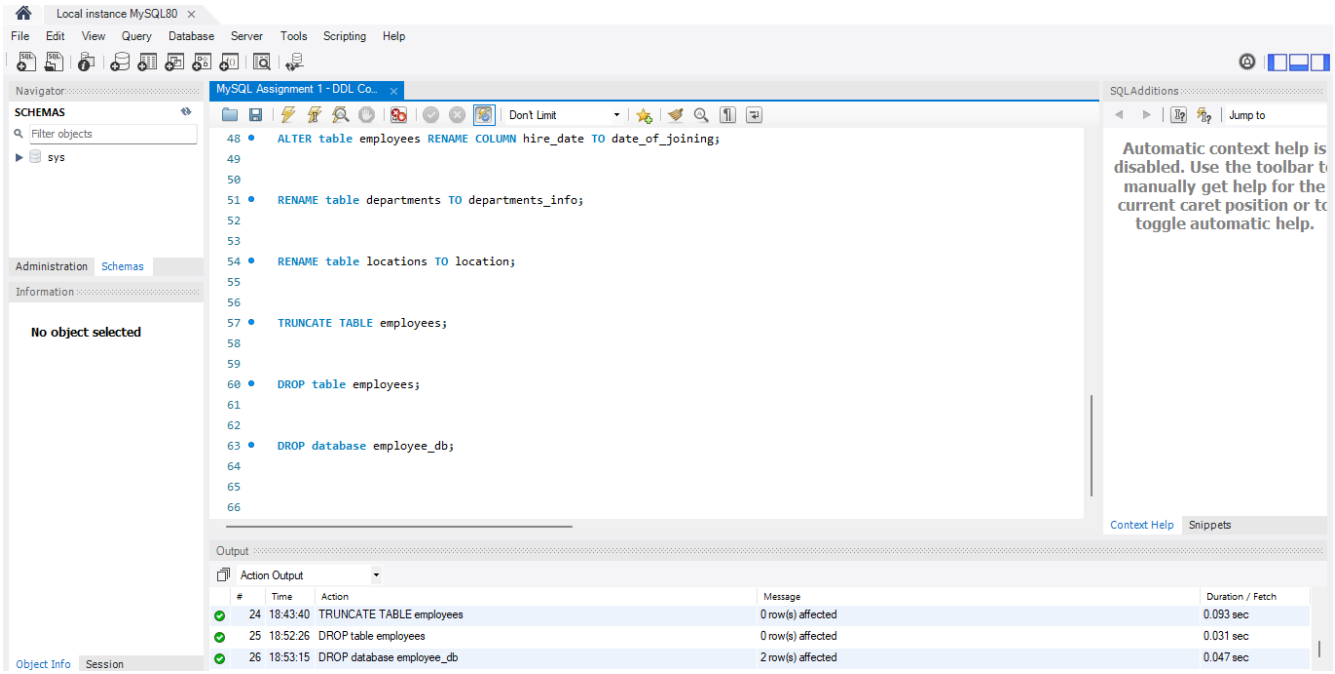
- Renamed departments table to department info using rename command
  RENAME table departments TO departments_info;
- Renamed table locations to location
  RENAME table locations TO location;

4. Table Truncation (TRUNCATE):

- To truncate employess table use Truncate command.
  TRUNCATE TABLE employees;

5. Database & Table Dropping (DROP):

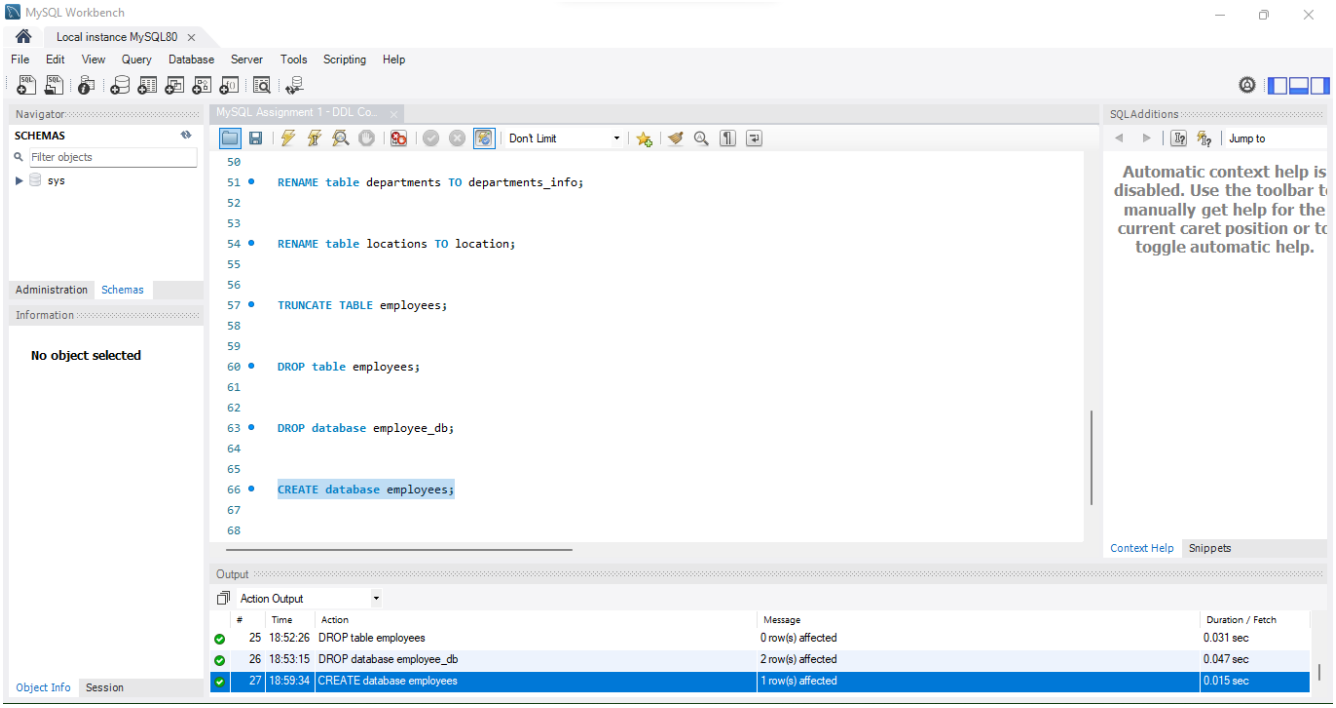
- To drop a table or database, use the drop command.
  For dropping employee table: DROP table employees;
  For dropping database: DROP database employee_db;



CONSTRAINTS :

1. Database Recreation:

New employees database was created



2. Departments Table:

- since we are using department_id as primary key, it will be uniquely identified.
- Added UNIQUE and NOT NULL constraints to department_name to avoid duplicate and null entries respectively.

  CREATE table departments(department_ID INT PRIMARY KEY, department_name VARCHAR(100) UNIQUE NOT NULL);

```
71 • CREATE table departments(
72   department_ID INT PRIMARY KEY,
73   department_name VARCHAR(100) UNIQUE NOT NULL
74   );
75
76
77 • SELECT * FROM departments;
78
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

department_ID	department_name
NULL	NULL

3. Location Table:

- To automatically increment unique identifiers AUTO_INCREMENT is used.
- As location ID is a primary key unique and not null values are a given. For location name UNIQUE and NOT NULL constraints are used.

```
80  CREATE table locations(
81  location_ID INT PRIMARY KEY AUTO_INCREMENT,
82  location VARCHAR(30) NOT NULL UNIQUE
83  );
84
85  SELECT * FROM locations;
86
```



4. Employees Table:

- Employee _ID is used as a identifier which is a primary key
- As a restriction to ensure that the employee's name is always provided, we us NOT NULL constraint.
- Enum datatype is used to limit the acceptable values for the "gender" field to only 'M' or 'F'.
- Used CHECK to to ensure that the employee's age is 18 or above.
- We use DEFAULT and CURRENT_DATE to automatically assign the current date to the "hire_date" field if not specified.
- WE use primary keys and foreign keys to establish links between the "department_id" and "location_id" fields in the "employees" table and their respective tables.

```sql
CREATE table employees(
employee_ID INT PRIMARY KEY,
employee_name VARCHAR(50) NOT NULL,
gender enum('M','F'),
age INT CHECK(age>=18),
hire_date DATE DEFAULT (CURRENT_DATE),
designation VARCHAR(100),
department_ID INT,
FOREIGN KEY (department_ID) REFERENCES departments(department_ID),
location_ID INT,
FOREIGN KEY (location_ID) REFERENCES locations(location_ID),
salary DECIMAL(10,2)
);

SELECT * FROM employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employee_ID	employee_name	gender	age	hire_date	designation	department_ID	location_ID	salary
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL